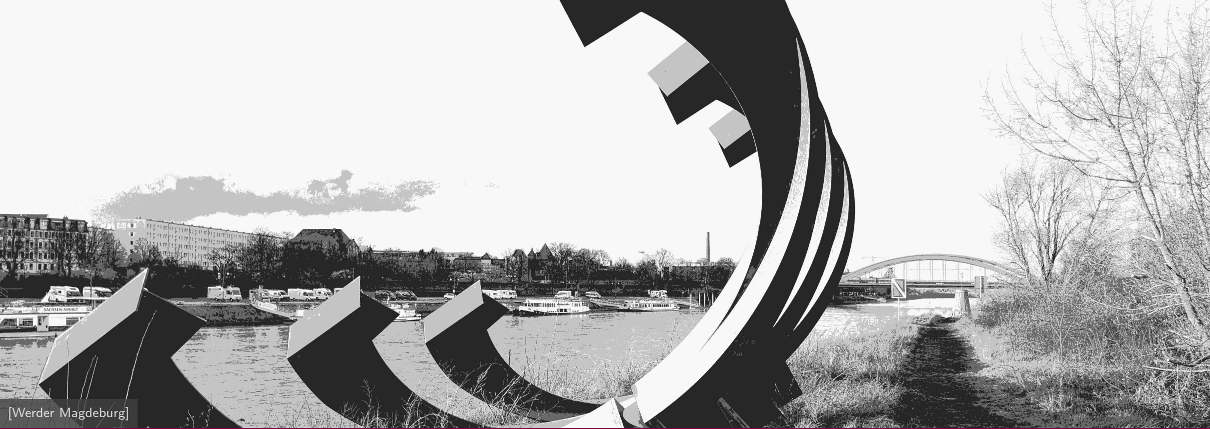


[Werder Magdeburg]

torte: Reproducible Feature-Model Experiments à la Carte

ICSE 2026 — April 12–18 — Rio de Janeiro, Brazil

Elias Kuiter[1]

University of Magdeburg[1], Germany

OTTO VON GUERICKE UNIVERSITÄT MAGDEBURG

Implementing Configurable Software Systems

A Configurable Graph

```
class Node {
  #ifdef LABELED
    std::string label;
  #endif
  #ifdef COLORED
    std::string color;
  #endif
};

class Edge {
  #ifdef DIRECTED
    Node from, to;
  #elif UNDIRECTED && HYPER
    std::set<Node> nodes;
  #endif
};
```

Product Line Implementation
(here: C++ with C preprocessor)

Implementing Configurable Software Systems

A Configurable Graph

```
class Node {
  #ifdef LABELED
    std::string label;
  #endif
  #ifdef COLORED
    std::string color;
  #endif
};

class Edge {
  #ifdef DIRECTED
    Node from, to;
  #elif UNDIRECTED && HYPER
    std::set<Node> nodes;
  #endif
};
```

Product Line Implementation
(here: C++ with C preprocessor)

#define
LABELED
DIRECTED

Configuration

Labeled Directed

```
class Node {
  std::string label;
};

class Edge {
  Node from, to;
};
```

Product Implementation

Implementing Configurable Software Systems

A Configurable Graph

```
class Node {
  #ifdef LABELED
    std::string label;
  #endif
  #ifdef COLORED
    std::string color;
  #endif
};

class Edge {
  #ifdef DIRECTED
    Node from, to;
  #elif UNDIRECTED && HYPER
    std::set<Node> nodes;
  #endif
};
```

Product Line Implementation
(here: C++ with C preprocessor)

#define LABELED DIRECTED

#define COLORED UNDIRECTED HYPER

Configuration

Labeled Directed

```
class Node {
  std::string label;
};

class Edge {
  Node from, to;
};
```

Colored Undirected Hyper

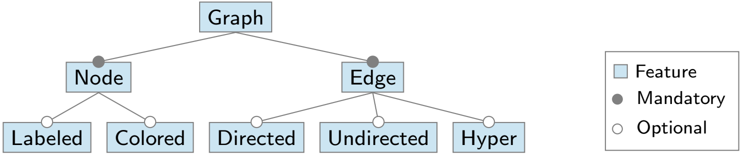
```
class Node {
  std::string color;
};

class Edge {
  std::set<Node> nodes;
};
```

Product Implementation

Modeling Features and their Dependencies

Feature Models

- tree models **features**
- cross-tree **constraints** model dependencies
- solver-based **analyses** can be used to understand the configuration space better

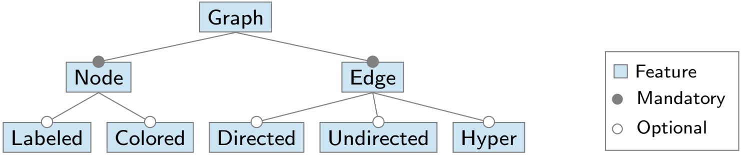


$\neg(\mathit{Directed} \land \mathit{Undirected})$	**feature conflict**
$\mathit{Hyper} \rightarrow \mathit{Undirected}$	**missing implementation**
$\mathit{Directed} \not\leftrightarrow (\mathit{Undirected} \land \mathit{Hyper})$	**well-defined #ifdef**

Modeling Features and their Dependencies

Feature Models

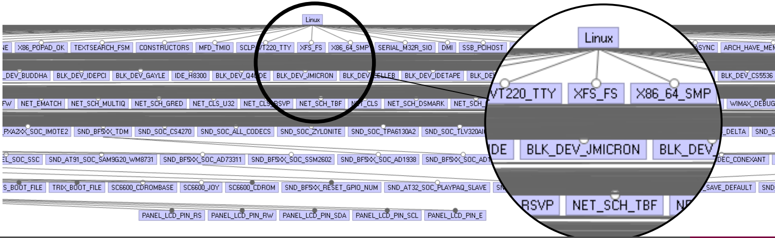
- tree models **features**
- cross-tree **constraints** model dependencies
- solver-based **analyses** can be used to understand the configuration space better



$\neg(\mathit{Directed} \land \mathit{Undirected})$	**feature conflict**
$\mathit{Hyper} \rightarrow \mathit{Undirected}$	**missing implementation**
$\mathit{Directed} \not\leftrightarrow (\mathit{Undirected} \land \mathit{Hyper})$	**well-defined #ifdef**

The Linux Kernel

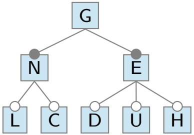
- ≈ 20000 features [2025]
- $> 10^{1600}$ products [2025]
- 114 dead features [2013]
- 151 reverse dependency bugs [2019]



Feature-Model Analysis – Background

Feature-Model Analysis – Background

Feature Model



$$\neg(D \wedge U)$$
$$H \rightarrow U$$
$$D \not\leftrightarrow (U \wedge H)$$

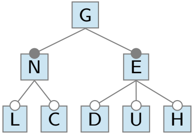
Feature-Model Analysis – Background

Model

ask

interactive

Feature Model

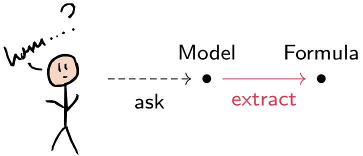


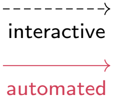
$$\neg(D \wedge U)$$
$$H \rightarrow U$$
$$D \not\leftrightarrow (U \wedge H)$$

Core Feature F?

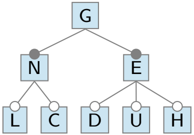
#Configurations?

Feature-Model Analysis – Background





Feature Model



$\neg(D \wedge U)$
$H \rightarrow U$
$D \not\leftrightarrow (U \wedge H)$

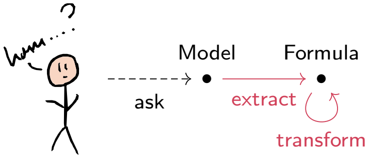
Formula

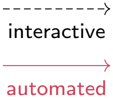
$$
\begin{aligned}
& G \\
\wedge\ & (N \leftrightarrow G) \wedge (E \leftrightarrow G) \\
\wedge\ & ((L \vee C) \rightarrow N) \\
\wedge\ & ((D \vee U \vee H) \rightarrow E) \\
\wedge\ & \neg(D \wedge U) \wedge (H \rightarrow U) \\
\wedge\ & (D \not\leftrightarrow (U \wedge H))
\end{aligned}
$$

Core Feature F?

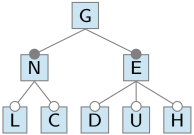
#Configurations?

Feature-Model Analysis – Background





Feature Model



$\neg(D \land U)$
$H \rightarrow U$
$D \not\leftrightarrow (U \land H)$

Formula

$$
\begin{aligned}
& G \\
\land\ & (N \leftrightarrow G) \land (E \leftrightarrow G) \\
\land\ & ((L \lor C) \rightarrow N) \\
\land\ & ((D \lor U \lor H) \rightarrow E) \\
\land\ & \neg(D \land U) \land (H \rightarrow U) \\
\land\ & (D \not\leftrightarrow (U \land H))
\end{aligned}
$$

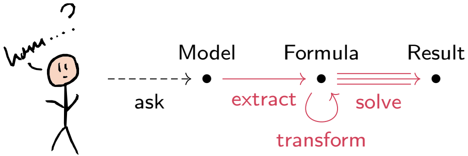
CNF

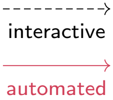
$$
\begin{aligned}
\{ & \{G\}, \{\neg N, G\}, \\
& \{N, \neg G\}, \{\neg E, G\}, \\
& \{E, \neg G\}, \{\neg L, N\}, \\
& \{\neg C, N\}, \{\neg D, E\}, \\
& \{\neg U, E\}, \{\neg H, E\}, \\
& \{\neg D, \neg U\}, \{\neg H, U\}, \\
& \{\{D, U\}, \{D, H\}, \\
& \{\neg D, \neg U, \neg H\}\}\}
\end{aligned}
$$

Core Feature F?

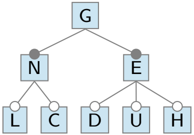
#Configurations?

Feature-Model Analysis – Background





Feature Model



$$\neg(D \land U)$$
$$H \rightarrow U$$
$$D \not\leftrightarrow (U \land H)$$

Formula

$$\begin{aligned} & G \\ \land\ & (N \leftrightarrow G) \land (E \leftrightarrow G) \\ \land\ & ((L \lor C) \rightarrow N) \\ \land\ & ((D \lor U \lor H) \rightarrow E) \\ \land\ & \neg(D \land U) \land (H \rightarrow U) \\ \land\ & (D \not\leftrightarrow (U \land H)) \end{aligned}$$

CNF

$$\begin{aligned} \{ & \{G\}, \{\neg N, G\}, \\ & \{N, \neg G\}, \{\neg E, G\}, \\ & \{E, \neg G\}, \{\neg L, N\}, \\ & \{\neg C, N\}, \{\neg D, E\}, \\ & \{\neg U, E\}, \{\neg H, E\}, \\ & \{\neg D, \neg U\}, \{\neg H, U\}, \\ & \{\{D, U\}, \{D, H\}, \\ & \{\neg D, \neg U, \neg H\}\}\} \end{aligned}$$

Core Feature F?

$$SAT(\Phi \land \neg F)$$

Core Features

$$\{G, N, E\}$$

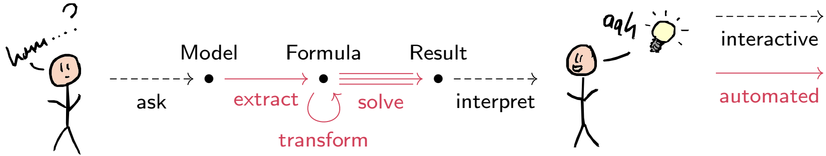
#Configurations?

$$\#SAT(\Phi)$$

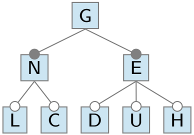
#Configurations

8

Feature-Model Analysis – Background



Feature Model



$\neg(D \wedge U)$
$H \rightarrow U$
$D \not\leftrightarrow (U \wedge H)$

Formula

$$G \wedge (N \leftrightarrow G) \wedge (E \leftrightarrow G) \wedge ((L \vee C) \rightarrow N) \wedge ((D \vee U \vee H) \rightarrow E) \wedge \neg(D \wedge U) \wedge (H \rightarrow U) \wedge (D \not\leftrightarrow (U \wedge H))$$

CNF

$$\{\{G\}, \{\neg N, G\}, \{N, \neg G\}, \{\neg E, G\}, \{E, \neg G\}, \{\neg L, N\}, \{\neg C, N\}, \{\neg D, E\}, \{\neg U, E\}, \{\neg H, E\}, \{\neg D, \neg U\}, \{\neg H, U\}, \{\{D, U\}, \{D, H\}, \{\neg D, \neg U, \neg H\}\}\}$$

Core Feature F?

$SAT(\Phi \wedge \neg F)$

Core Features

$\{G, N, E\}$

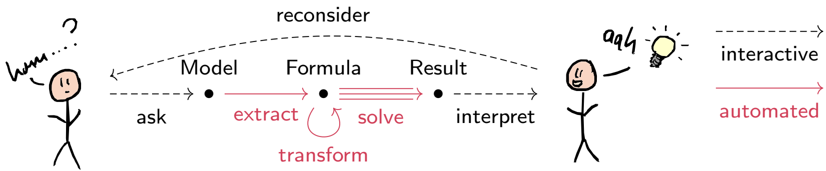
#Configurations?

$\#SAT(\Phi)$

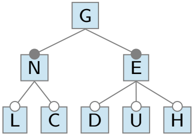
#Configurations

8

Feature-Model Analysis – Background



Feature Model



$\neg(D \wedge U)$

$H \rightarrow U$

$D \not\leftrightarrow (U \wedge H)$

Formula

$$G \wedge (N \leftrightarrow G) \wedge (E \leftrightarrow G) \wedge ((L \vee C) \rightarrow N) \wedge ((D \vee U \vee H) \rightarrow E) \wedge \neg(D \wedge U) \wedge (H \rightarrow U) \wedge (D \not\leftrightarrow (U \wedge H))$$

CNF

$$\{\{G\}, \{\neg N, G\}, \{N, \neg G\}, \{\neg E, G\}, \{E, \neg G\}, \{\neg L, N\}, \{\neg C, N\}, \{\neg D, E\}, \{\neg U, E\}, \{\neg H, E\}, \{\neg D, \neg U\}, \{\neg H, U\}, \{\{D,U\}, \{D, H\}, \{\neg D, \neg U, \neg H\}\}\}$$

Core Feature F?

$SAT(\Phi \wedge \neg F)$

Core Features

$\{G, N, E\}$

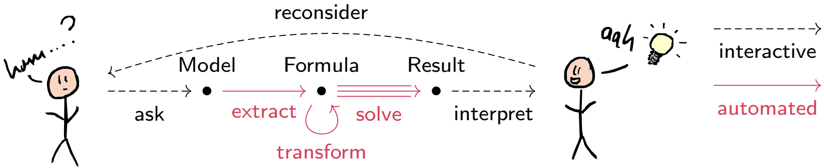
#Configurations?

$\#SAT(\Phi)$

#Configurations

8

Feature-Model Analysis – Background



Feature Model

G
N E
L C D U H

$\neg(D \wedge U)$
$H \rightarrow U$
$D \not\leftrightarrow (U \wedge H)$

Formula

$$\begin{aligned} & G \\ \wedge\ & (N \leftrightarrow G) \wedge (E \leftrightarrow G) \\ \wedge\ & ((L \vee C) \rightarrow N) \\ \wedge\ & ((D \vee U \vee H) \rightarrow E) \\ \wedge\ & \neg(D \wedge U) \wedge (H \rightarrow U) \\ \wedge\ & (D \not\leftrightarrow (U \wedge H)) \end{aligned}$$

CNF

$$\begin{aligned} \{ & \{G\}, \{\neg N, G\}, \\ & \{N, \neg G\}, \{\neg E, G\}, \\ & \{E, \neg G\}, \{\neg L, N\}, \\ & \{\neg C, N\}, \{\neg D, E\}, \\ & \{\neg U, E\}, \{\neg H, E\}, \\ & \{\neg D, \neg U\}, \{\neg H, U\}, \\ & \{\{D, U\}, \{D, H\}, \\ & \{\neg D, \neg U, \neg H\}\}\} \end{aligned}$$

Core Feature F?

$SAT(\Phi \wedge \neg F)$

Core Features

$\{G, N, E\}$

#Configurations?

$\#SAT(\Phi)$

#Configurations

8

interactive product configuration · anomaly detection and explanation · feature-model evolution and slicing · modularization · implicit constraints · code parsing · dead code analysis · code simplification · type checking · model checking · formal verification · static code analysis · dataflow analysis · testing · t-wise sampling · uniform random sampling · economical estimations · refactoring · . . .

Feature-Model Analysis – Open Challenges

Feature-Model Analysis – Open Challenges

History Extraction

BusyBox (20 years) axTLS (14 years) Buildroot (17 years) EmbToolkit (6 years) Freetz-NG (19 years) L4Re (9 years) toybox (19 years) uClibc (10 years) **Linux (24 years)** uClibc-ng (10 years) coreboot u-boot Crosstool-NG Soletta FreeWRT QEMU RIOT Unikraft Zephyr esp-idf sdk-nrf . . .

KConfig **evolves**/**diverges** ⇒ no consistent histories

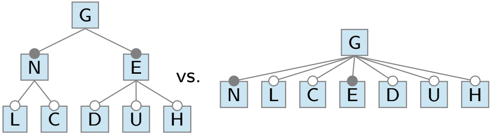
Feature-Model Analysis – Open Challenges

History Extraction

BusyBox (20 years) axTLS (14 years) Buildroot (17 years) EmbToolkit (6 years) Freetz-NG (19 years) L4Re (9 years) toybox (19 years) uClibc (10 years) **Linux (24 years)** uClibc-ng (10 years) coreboot u-boot Crosstool-NG Soletta FreeWRT QEMU RIOT Unikraft Zephyr esp-idf sdk-nrf ...

KConfig **evolves**/**diverges** ⇒ no consistent histories

Hierarchy Extraction



extracted models are **flat** ⇒ loses developer intent

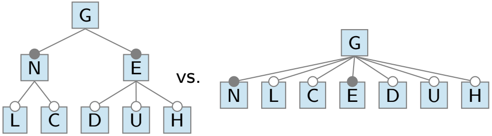
Feature-Model Analysis – Open Challenges

History Extraction

BusyBox (20 years) axTLS (14 years) Buildroot (17 years) EmbToolkit (6 years) Freetz-NG (19 years) L4Re (9 years) toybox (19 years) uClibc (10 years) **Linux (24 years)** uClibc-ng (10 years) coreboot u-boot Crosstool-NG Soletta FreeWRT QEMU RIOT Unikraft Zephyr esp-idf sdk-nrf ...

KConfig **evolves**/**diverges** ⇒ no consistent histories

Hierarchy Extraction



extracted models are **flat** ⇒ loses developer intent

Automation and Reproducibility

`913bf31 · 10 years ago` outdated dependencies

`linked, for GNU/Linux 2.0.0,` precompiled binaries

experiments are **rarely reproduced**

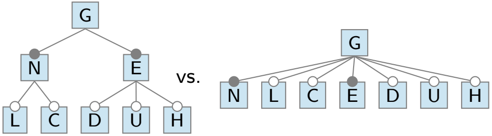
Feature-Model Analysis – Open Challenges

History Extraction

BusyBox (20 years) axTLS (14 years) Buildroot (17 years) EmbToolkit (6 years) Freetz-NG (19 years) L4Re (9 years) toybox (19 years) uClibc (10 years) **Linux (24 years)** uClibc-ng (10 years) coreboot u-boot Crosstool-NG Soletta FreeWRT QEMU RIOT Unikraft Zephyr esp-idf sdk-nrf . . .

KConfig **evolves**/**diverges** ⇒ no consistent histories

Hierarchy Extraction



extracted models are **flat** ⇒ loses developer intent

Automation and Reproducibility

913bf31 · 10 years ago outdated dependencies

`linked, for GNU/Linux 2.0.0,` precompiled binaries

experiments are **rarely reproduced**

Reusability and Interoperability

handcrafted experiments, limited **comparability**

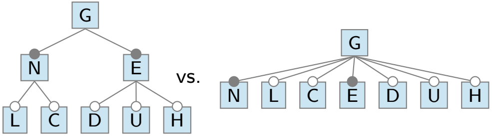
Feature-Model Analysis – Open Challenges

History Extraction

BusyBox (20 years) axTLS (14 years) Buildroot (17 years) EmbToolkit (6 years) Freetz-NG (19 years) L4Re (9 years) toybox (19 years) uClibc (10 years) **Linux (24 years)** uClibc-ng (10 years) coreboot u-boot Crosstool-NG Soletta FreeWRT QEMU RIOT Unikraft Zephyr esp-idf sdk-nrf . . .

KConfig **evolves**/**diverges** $\Rightarrow$ no consistent histories

Hierarchy Extraction



extracted models are **flat** $\Rightarrow$ loses developer intent

Automation and Reproducibility

913bf31 · 10 years ago outdated dependencies

`linked, for GNU/Linux 2.0.0,` precompiled binaries

experiments are **rarely reproduced**

Reusability and Interoperability

handcrafted experiments, limited **comparability**

Ease of Use

C(++) Scala Rust Java Bash Python · · ·

complex orchestration of heterogeneous tools


Introducing: **torte**!

torte – An Experimentation Platform For Feature-Model Analysis

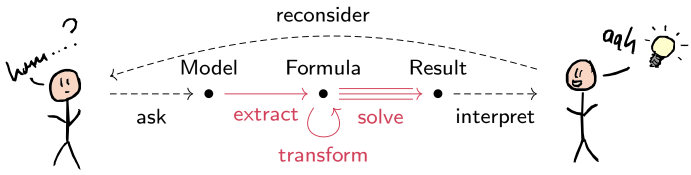
torte – An Experimentation Platform For Feature-Model Analysis

torte 🍰

Roll your own experiments!

/ekuiter/🍰

torte – An Experimentation Platform For Feature-Model Analysis

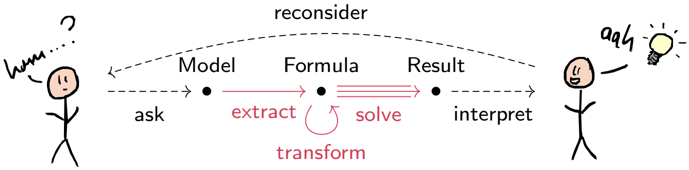


torte 🍰

Roll your own experiments!

/ekuiter/🍰

torte – An Experimentation Platform For Feature-Model Analysis



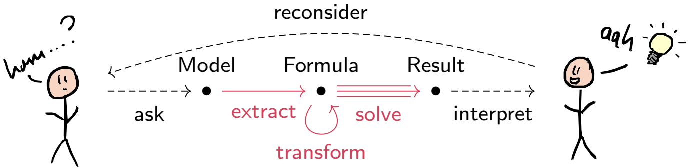
Extraction
KConfigReader KClause ConfigFix Hierarchy extractor Benchmark models

torte 🍰

Roll your own experiments!

/ekuiter/🍰

torte – An Experimentation Platform For Feature-Model Analysis



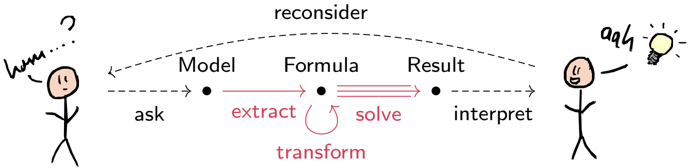
Extraction	Transformation
KConfigReader	FeatureIDE
KClause	FeatJAR
ConfigFix	clausy
Hierarchy extractor	Z3
Benchmark models	CadiBack

torte 🍰

Roll your own experiments!

/ekuiter/🍰

torte – An Experimentation Platform For Feature-Model Analysis



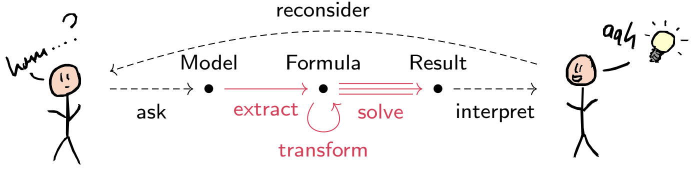
Extraction	Transformation	Solving
KConfigReader	FeatureIDE	53 SAT
KClause	FeatJAR	14 #SAT
ConfigFix	clausy	SATGraf
Hierarchy extractor	Z3	Jupyter notebooks
Benchmark models	CadiBack	Web dashboard

torte 🍰

Roll your own experiments!

/ekuiter/🍰

torte – An Experimentation Platform For Feature-Model Analysis



Extraction	Transformation	Solving
KConfigReader	FeatureIDE	53 SAT
KClause	FeatJAR	14 #SAT
ConfigFix	clausy	SATGraf
Hierarchy extractor	Z3	Jupyter notebooks
Benchmark models	CadiBack	Web dashboard
Experimentation Platform		
Docker containerization	Job parallelization	Stage architecture
Continuous integration	Profiling system	Experiment DSL
Reproduction packages	Resource limiting	Remote execution

torte 🍰

Roll your own experiments!

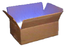
/ekuiter/🍰

torte – Short Demonstration

torte – Short Demonstration

How configurable is BusyBox?

torte – Short Demonstration

How configurable is BusyBox? 

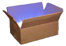
Self-Executing One-Liner

```
curl -sL https://elias-kuiter.de/torte/ | sh
```

Requirements Docker, Git, curl, and GNU tools

torte – Short Demonstration

```bash
#!/bin/bash
TORTE_REVISION=7a65a22
TIMEOUT=10
JOBS=4
[[ $TOOL != torte ]] && builtin source /dev/stdin <<<"$(curl -fsSL \
https://raw.githubusercontent.com/ekuiter/torte/7a65a22/torte.sh)"

experiment-systems() {
  add-busybox-kconfig-history --from 1_36_0 --to 1_36_1
}

experiment-stages() {
  clone-systems; read-statistics; extract-kconfig-models

  # Serializes the feature model of BusyBox as UVL
  transform-model-with-featjar \
  --transformer transform-model-to-uvl-with-featureide \
  --output-extension uvl
  transform-model-to-dimacs

  # Counts the number of valid configurations of BusyBox
  solve-sharp-sat --timeout "$TIMEOUT" --jobs "$JOBS"
}
```

How configurable is BusyBox? 

Self-Executing One-Liner

```
curl -sL https://elias-kuiter.de/torte/ | sh
```

Requirements Docker, Git, curl, and GNU tools

torte – Short Demonstration

```bash
#!/bin/bash
TORTE_REVISION=7a65a22
TIMEOUT=10
JOBS=4
[[ $TOOL != torte ]] && builtin source /dev/stdin <<<"$(curl -fsSL \
https://raw.githubusercontent.com/ekuiter/torte/7a65a22/torte.sh)"

experiment-systems() {
  add-busybox-kconfig-history --from 1_36_0 --to 1_36_1
}

experiment-stages() {
  clone-systems; read-statistics; extract-kconfig-models

  # Serializes the feature model of BusyBox as UVL
  transform-model-with-featjar \
  --transformer transform-model-to-uvl-with-featureide \
  --output-extension uvl
  transform-model-to-dimacs

  # Counts the number of valid configurations of BusyBox
  solve-sharp-sat --timeout "$TIMEOUT" --jobs "$JOBS"
}
```

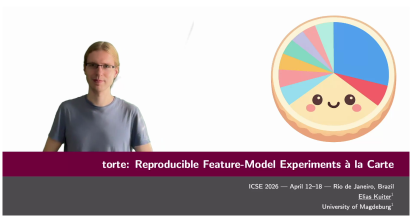
How configurable is BusyBox?

Self-Executing One-Liner

```
curl -sL https://elias-kuiter.de/torte/ | sh
```

Requirements Docker, Git, curl, and GNU tools

Demo Video



Experiences with torte

Experiences with torte

Tseitin or not Tseitin? The Impact of CNF Transformations on Feature-Model Analyses

ASE'22

How Configurable Is the Linux Kernel? Analyzing Two Decades of Feature-Model History

TOSEM'25

Investigating the Effects of T-Wise Interaction Sampling for Vulnerability Discovery in Highly-Configurable Software Systems

SPLC'25

Can SAT Solvers Keep Up With the Linux Kernel's Feature Model?

ICSE '26

Experiences with torte

- 7 Zenodo artifacts
- 3 ACM "Reusable" badges
- part of 1 best artifact award
- 6 student theses and projects

Experiences with torte

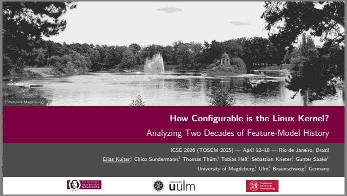
- 7 Zenodo artifacts
- 3 ACM "Reusable" badges
- part of 1 best artifact award
- 6 student theses and projects

New: Review of Tool Ecosystem Beyond Torte

torte 🍰 Research

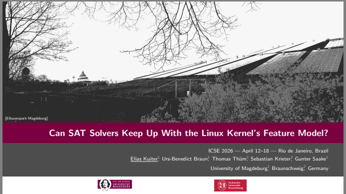
Conclusion

Journal-First Paper



How Configurable Is the Linux Kernel?

Research Paper



Can SAT Solvers Keep Up With the Linux Kernel's Feature Model?

Demo Paper

Image Credits

- **BusyBox logo**, BusyBox developers [https://commons.wikimedia.org/wiki/File:BusyBoxLogo.png]
- **LEGO minifigures**, Thomas Thüm [https://github.com/TUBS-ISF/Course-on-Software-Product-Lines/tree/main/pics/lego]
- **TikZpingus**, Florian Sihler [https://github.com/EagleoutIce/tikzpingus]

All images were used with permission. The slides were created with FancyBeamer and draw.io.